1. ecdl-tensor-ic: Rust port of the Go ecdltensor prototype — port verification

Run id: run-20261030T0017Z (UTC). Base commit: main a6d84669a (branch claude/ecdl-tensor-ic-rust-port-20261029). Go source: carrier branch research/ecdl-tensor-go-source-20261029, commit 1c29f66. Host: Claude Code cloud container (linux/amd64), no isolation; rustc 1.97.0, go 1.24.7. Kind: correctness and structure check of a code port. **Not a performance measurement; no leaderboard, benchmarks/ or validation-gate row.**

1.1. What ran

- `cargo test --release --locked` in `crates/ecdl-tensor-ic`: 21 tests passed, 0 failed, 0 ignored (19 ported Go tests: 7 in `tests/ecdl_test.rs`, 12 in `tests/gadget_test.rs`; plus 2 unit tests). Wall 24.6 s. The Go suite (`go test ./...`, go 1.24.7) passed in 43.2 s on the same host (single runs, observation only).
- The Go test names (`func TestXxx`) and the Rust `// Go: TestXxx` markers match one to one (scripted diff, empty).
- `cargo fmt --check` and `cargo clippy --locked --all-targets -- -D warnings`: clean.
- `ecdlscale` (Rust) vs `go run ./cmd/ecdlscale`: byte-identical output (`diff`, empty). Files: `ecdlscale-go.txt`, `ecdlscale-rust.txt`.
- `ecdldemo -bits 14 -m 2` and `-bits 8 -m 2 -gadgets`, Go and Rust, one run each. Both recover the secret and verify $[\xi]P = Q$. The network shapes are identical: 87 edges, 4 vertices, max degree 87 (whole-point); 7591 edges, 606 vertices, max degree 130 (gadget). The curves differ because Go's `math/rand` stream is not reproduced, so the times (Rust 24.0 s and 5.4 s; Go 24.5 s and 20.0 s) are not comparable and support no speed statement.

1.2. Structure (recorded values)

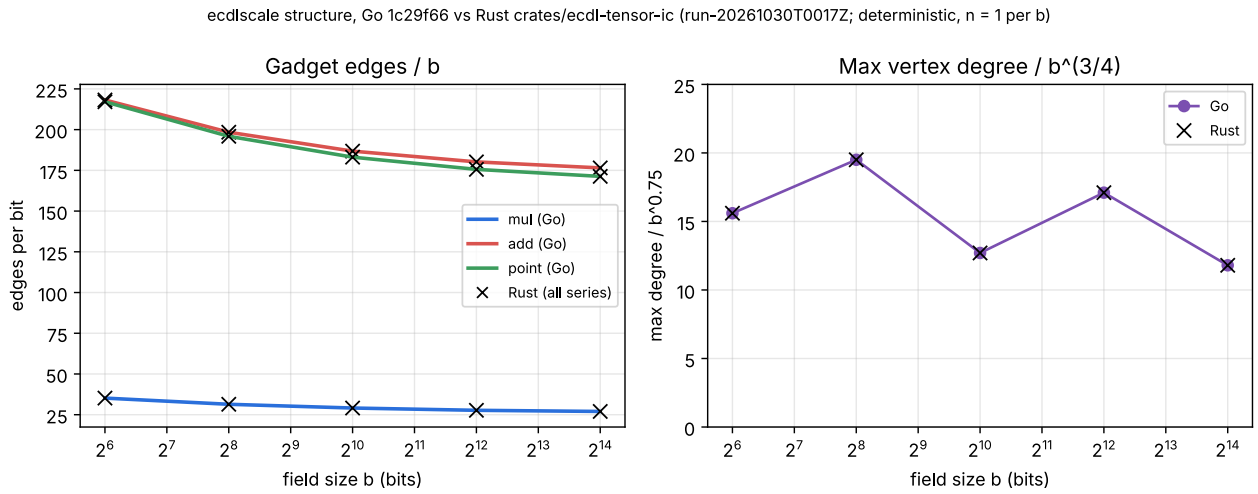


Figure 1: Edges per bit and max degree / $b^{\frac{3}{4}}$ of the mul, complete-add and point (validity + membership) gadget networks, $p = 2^{b-1} + 1$, from `ecdlscale` in Go (lines) and Rust (crosses). Deterministic; one build per b.

b	mul e/b	add e/b	point e/b	max degree	deg / b^0.75
64	35.2	218.2	217.1	352	15.6
256	31.4	198.4	195.9	1248	19.5
1024	29.1	186.8	183.1	2304	12.7
4096	27.7	180.2	175.6	8768	17.1
16384	27.0	176.5	171.3	17024	11.8

1.3. Semantic checks covered by the ported tests

- Exhaustive gadget tests (every hint enumerates every bit pattern it writes) leave exactly one assignment, equal to the correct value: `mul_check` for all pairs at p = 11, 13 and sampled at p = 101; `lin_check`/`range_check`, validity, validity + membership at 4 bits; `complete_add` on all pairs with O or equal x on a 4-bit prime-order curve (289 pairs, 65 special at p = 13) and on $y^2 = x^3 + x$ over F_{13} (three y = 0 points).
- Gadget relation counts equal whole-point counts for m = 2 (6 targets, 2 reached only through special cases) and m = 3 (8 targets, 4 special), targets including R = O.
- `solve` recovers ξ on 14 toy curves (e.g. b = 10, m = 2, p = 823) and with the gadget network at p = 53.

- Independent verifier agents also compared NTT parameters (t, n, L, L1, L2, Q, WQ, WE, ω, L^{-1}) and GadgetShapes for 17 moduli up to b = 521 between the real Go code and the port: identical.

1.4. Differences from Go

RNG: `StdRng` seeded with the Go seeds instead of `math/rand`. Primality for Q: Miller–Rabin on 30 fixed bases instead of `ProbablyPrime(30)` (no Baillie–PSW step); same Q at every size checked. API: `Vertex` enum, `Counter` trait, `Var` = `Rc<RefCell<…>>`, `Solver::solve_with_stats`, `Builder::hint_nil`.

1.5. Limits

Toy sizes only. No speedup: every counter here is exponential and the [Ope26] contraction algorithm is not implemented; the prototype costs about r per relation, worse than Pollard rho. The gadgets, the NTT construction and the affine factor-base hash are our own construction, not the paper's. It cannot attack deployed curves (P-256, Ed25519).